Dimensionality Reduction in NLP: Theoretical Foundations, Methods, and Applications

Stefan Pop

April 20, 2026

Abstract

Natural Language Processing (NLP) relies on high-dimensional data structures, where vocabularies of thousands of words create sparse, computationally expensive matrices. This report synthesizes and compares three fundamental dimensionality reduction algorithms used to extract underlying semantic meaning from text: Principal Component Analysis (PCA), Latent Semantic Analysis (LSA), and t-Distributed Stochastic Neighbor Embedding (t-SNE). We observe tthe way PCA establishes the mathematical baseline of variance maximization and covariance diagonalization. We then explore how LSA technically adapts this via Singular Value Decomposition (SVD) to map terms and documents into a shared latent space, solving issues like synonymy and polysemy. Finally, we contrast these linear methods with t-SNE, a non-linear probabilistic approach that addresses the "crowding problem" to effectively visualize high-dimensional word embeddings. Together, these three methods constitute a complete pipeline for representing, compressing, retrieving, and visualizing linguistic data.

Contents

1 Introduction and Historical Context

Early automatic indexing and information retrieval (IR) systems were plagued by a fundamental linguistic problem: users rarely query using the exact same words authors use to index their documents [1]. This literal term-matching approach fails due to two widespread language properties: **synonymy** and **polysemy** [1]. Synonymy refers to multiple words representing a single concept (e.g., "car" and "automobile"), which artificially lowers a system's *recall*—its ability to find all relevant documents [1]. Polysemy refers to a single word holding multiple meanings depending on context (e.g., "chip" in electronics vs. food), which actively damages a system's *precision*—its ability to avoid returning irrelevant documents [1].

Empirical studies have made the severity of synonymy concrete: two independent users choose the same primary keyword for a single well-known object less than 20% of the time [1]. Historically, attempts to solve these issues relied on intellectual or automatic term expansion, or the construction of thesauruses [1]. However, these were computationally expensive, hard to maintain, and unreliable when polysemous terms were introduced.

The major paradigm shift in NLP occurred when researchers began treating the unreliability of observed term-document association as a **statistical problem**: they theorized that there is an underlying "latent" semantic structure obscured by the randomness of human word choice [1]. This shift necessitated the adoption of dimensionality reduction algorithms to mathematically compress the vocabulary space, filter out linguistic noise, and extract true semantic meaning—exactly the challenge addressed by LSA, PCA, and t-SNE.

2 The Bag-of-Words Model and the Term-Document Matrix

The main cause of the dimensionality reduction problem in NLP is understanding the data structure the algorithms actually work on. That being said, we will look in more depth to better understand this concept

2.1 The Bag-of-Words (BoW) Representation

The dominant paradigm for representing text computationally is the **Bag-of-Words (BoW)** model. Under BoW, a document is represented as a vector of word counts, completely disregarding grammar and word order. A corpus of d documents with a vocabulary of t unique terms is thus encoded as a $t \times d$ matrix X, where entry X_{ij} records how many times term i appears in document j [1].

The fundamental problem is immediately apparent: for a realistic corpus, the vocabulary t can reach tens of thousands of terms, making X an enormous, extremely **sparse** matrix (most entries are zero). This high dimensionality creates computational bottlenecks in storage and retrieval, motivating compression.

2.2 TF-IDF Weighting

Raw term frequencies are a poor representation because common words like "the" or "is" appear in nearly every document and carry no discriminating semantic content. In

practice, the raw frequency values in X are replaced by **TF-IDF** (Term Frequency-Inverse Document Frequency) weights before applying any dimensionality reduction:

$$\text{TF-IDF}(i,j) = \underbrace{\text{tf}(i,j)}_{\text{local weight}} \times \underbrace{\log\left(\frac{d}{\text{df}(i)}\right)}_{\text{global weight}} \tag{1}$$

where $\text{tf}(i,j)$ is the frequency of term i in document j, d is the total number of documents, and $\text{df}(i)$ is the number of documents containing term i. This weighting scheme increases the meaning of more rare terms and diminishes the importance of more used (common) ones before the matrix is fed into SVD

2.3 Motivating Example: The Synonymy–Polysemy Failure

Table 1 reproduces a canonical example from Deerwester et al. [1] that concretely illustrates the failure of literal term matching. Three documents touch on overlapping topics (information retrieval and database theory), yet a query about "IDF in computer-based information look-up" will only return Documents 2 and 3 (which contain the query terms *computer* and *information*)—missing Document 1 entirely, despite its close relevance. Document 2 is returned *erroneously*: its use of the word "computer" refers to computer science theory, not information retrieval. This table is the direct motivation for LSA.

Table 1: A sample Term-by-Document matrix...

	access	*document*	*retrieval*	*information*	*theory*	*database*	*indexing*	*computer*	REL	MATCH
Doc 1	x	x	x			x	x		✓	—
Doc 2				x*	x			x*		✓
Doc 3			x	x*				x*	✓	✓

Query: "IDF in *computer*-based *information* look-up" (x* = term appears in query *and* document)

3 Theoretical Foundations: Principal Component Analysis (PCA)

To understand the mechanics of semantic dimensionality reduction, one must first establish the mathematical baseline of Principal Component Analysis (PCA). PCA is a non-parametric method for extracting relevant information from confusing datasets by reducing a complex dataset to a lower dimension, revealing hidden, simplified structures [2].

3.1 Change of Basis and the Goal of PCA

Let X be an $m \times n$ data matrix, where each column represents one observation and each row represents one measurement type. PCA seeks a linear transformation P such that $Y = PX$ yields a new representation where the covariance matrix of Y is *diagonal*. Formally, defining the rows of P as $\{p_1, \ldots, p_m\}$, the i-th principal component p_i is the

direction that captures the i-th largest variance in the data, under the constraint of mutual orthogonality.

3.2 Variance, Signal, and Noise

PCA operates on a fundamental assumption: directions within the data measurement space that contain the largest variances also contain the dynamics of interest [2]. In any dataset, noise is quantified relative to the strength of the actual signal, a relationship measured by the Signal-to-Noise Ratio (SNR):

$$\text{SNR} = \frac{\sigma^2_{\text{signal}}}{\sigma^2_{\text{noise}}} \tag{2}$$

A high SNR indicates high precision, meaning the primary direction of variance is the true signal, while the perpendicular spread represents random noise [2]. PCA mathematically hunts for this signal by drawing orthogonal axes—the *Principal Components*—through the dataset, rank-ordered by the amount of variance they capture [2].

3.3 The Covariance Matrix and Diagonalization

In high-dimensional spaces, PCA bypasses human visualization by calculating a Covariance Matrix (C_X), which captures the variance and redundancy between all possible pairs of measurements [2]:

$$C_X \equiv \frac{1}{n} X X^T \tag{3}$$

The diagonal entries of C_X encode the variance of each individual measurement type, while the off-diagonal entries encode the cross-correlations (redundancies) between pairs. The goal of PCA is to find a change of basis P such that the resulting covariance matrix C_Y is strictly diagonal [2]:

$$C_Y = P C_X P^T = \frac{1}{n} Y Y^T \tag{4}$$

By forcing all off-diagonal terms to zero, PCA mathematically guarantees that the new variables (principal components) are completely *decorrelated*, retaining only the most vital, rank-ordered variances along the diagonal. Based on the spectral theorem for symmetric matrices, P is set to be the matrix of eigenvectors of C_X [2].

3.4 Connection to SVD

Importantly, PCA is intimately related to **Singular Value Decomposition (SVD)**. Any $m \times n$ matrix X can be factored as:

$$X = U \Sigma V^T \tag{5}$$

where U and V are orthonormal matrices of left and right singular vectors, respectively, and Σ is a diagonal matrix of non-negative singular values in decreasing order. The connection to PCA is direct: the columns of U are the eigenvectors of $C_X = \frac{1}{n} X X^T$, and the singular values σ_k satisfy $\lambda_k = \sigma_k^2 / n$, where λ_k are the eigenvalues of C_X [2]. This relationship means that SVD is not just a computational tool—it *is* PCA.

4 Latent Semantic Analysis (LSA): The NLP Adaptation

Latent Semantic Analysis (LSA) adapts the linear algebraic principles of PCA specifically to the domain of Natural Language Processing. The core insight is that the same SVD factorization used to solve PCA can be applied to the term-document matrix to expose the hidden semantic structure that literal word-matching cannot access [1].

4.1 The LSA Model: SVD on the Term-Document Matrix

LSA begins with the (TF-IDF weighted) term-document matrix X of shape $t \times d$, where t is the vocabulary size and d the number of documents [1]. SVD decomposes this matrix as:

$$X = T_0 S_0 D_0^T \tag{6}$$

where [1]:

- T_0 $(t \times r)$ is the matrix of left singular vectors, one per *term*, with orthonormal columns.
- D_0 $(d \times r)$ is the matrix of right singular vectors, one per *document*, with orthonormal columns.
- S_0 $(r \times r)$ is the diagonal matrix of singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$, rank-ordered by magnitude.
- $r \leq \min(t, d)$ is the rank of X.

A crucial geometric interpretation: each singular value σ_k scales the k-th "topic axis" in the latent space. Large singular values correspond to dominant, recurring patterns of word co-occurrence—the semantic topics of the corpus—while small singular values correspond to idiosyncratic, noisy co-occurrences.

4.2 Dimensionality Reduction: The Rank-k Approximation

The core mechanism of LSA is truncating the decomposition by retaining only the k largest singular values and zeroing out the rest [1]:

$$\hat{X} = T_k S_k D_k^T \tag{7}$$

By the Eckart-Young-Mirsky theorem, $\hat{X}$ is the closest rank-k matrix to the original X in the Frobenius norm—it is the optimal least-squares approximation [1]. Since the original matrix X contains noise from arbitrary human word choice, this *imperfect* reconstruction is highly desirable [1]. Truncating to $k \approx 100$ dimensions has two key effects:

1. **Synonym merging:** Words like "car" and "automobile" that appear in similar document contexts will receive similar coordinate vectors in the latent space, even if they never co-occur directly. LSA learns that they are semantically equivalent from the co-occurrence statistics.
2. **Polysemy disambiguation:** The contribution of a polysemous word to the latent space is spread across multiple dimensions, each capturing a different sense, weighted by the surrounding context of each individual document.

4.3 The Shared Latent Space and Cosine Similarity

After truncation, both terms and documents are represented as coordinate vectors in the *same* k-dimensional latent space [1]. The scaled coordinates are:

$$\text{Term } i: \quad \hat{t}_i = T_k[i,:] \cdot S_k \tag{8}$$

$$\text{Document } j: \quad \hat{d}_j = D_k[j,:] \cdot S_k \tag{9}$$

Semantic similarity between any two objects (term-term, document-document, or term-document) is then measured by the **cosine similarity** of their coordinate vectors:

$$\text{sim}(a,b) = \frac{\hat{a} \cdot \hat{b}}{\|\hat{a}\| \cdot \|\hat{b}\|} \tag{10}$$

A value near 1 indicates near-identical semantic content; a value near 0 indicates unrelated content. Crucially, this can yield high similarity scores between objects that share *zero* literal words, directly solving the synonymy problem [1].

4.4 Query Processing in an IR System

To use LSA as an information retrieval engine, a new query q is represented as a sparse term-frequency vector x_q (same vocabulary as the corpus) and "folded in" to the latent space as a pseudo-document:

$$\hat{d}_q = x_q^T T_k S_k^{-1} \tag{11}$$

The system then ranks all documents by their cosine similarity to $\hat{d}_q$ and returns the top results [1]. Figure 1 illustrates this complete pipeline.

5 t-SNE: Non-linear Visualization and the Crowding Problem

While PCA and LSA are highly efficient linear methods, they suffer a significant limitation for *visualization*: mapping complex, non-linear semantic relationships down to two or three dimensions using a linear projection often results in chaotic, overlapping clouds that fail to preserve local semantic neighborhoods [3].

5.1 The Crowding Problem

In high-dimensional spaces, the available volume scales exponentially with the number of dimensions. A point at moderate distance in 100 dimensions can be faithfully represented at moderate distance in a 10-dimensional map. But when collapsing to 2D, there is simply insufficient geometric area: the algorithm is forced to place moderately distant points far too close together, creating spurious attractive forces that ultimately crush the entire dataset toward the center of the map [3]. This is the **crowding problem**, and it is a structural failure of linear projection, not a numerical artifact.

5.2 From SNE to t-SNE: Replacing Gaussians with Student-t

t-SNE is a refinement of Stochastic Neighbor Embedding (SNE) that solves the crowding problem through a specific distributional choice.

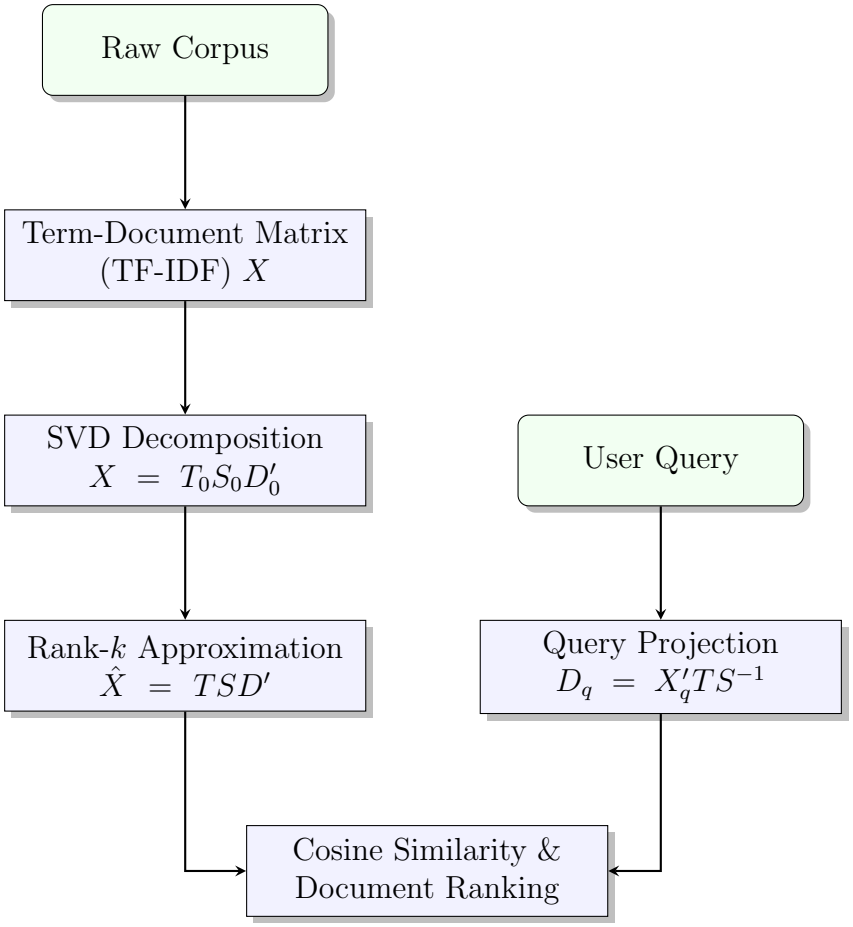

Figure 1: The end-to-end LSA information retrieval pipeline. A raw corpus is first converted to a TF-IDF weighted term-document matrix. SVD decomposes this matrix, and truncation to k dimensions yields the latent semantic space. Queries are projected into this space and ranked documents are returned by cosine similarity.

Step 1 — High-dimensional similarities. In the original space $\mathcal{X} = \{x_1, \ldots, x_n\}$, pairwise Euclidean distances are converted to *symmetrized* conditional probabilities using a Gaussian kernel [3]:

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq l} \exp(-\|x_k - x_l\|^2 / 2\sigma_i^2)} \tag{12}$$

The bandwidth σ_i for each point x_i is chosen via a binary search to match a user-specified **perplexity** $\mathrm{Perp}(P_i)$, defined using the Shannon entropy H of the resulting distribution:

$$\mathrm{Perp}(P_i) = 2^{H(P_i)}, \quad H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i} \tag{13}$$

Perplexity can be interpreted as a smooth measure of the effective number of neighbors; typical values range between 5 and 50 [3].

Step 2 — Low-dimensional similarities with Student-t. In the low-dimensional map $\mathcal{Y} = \{y_1, \ldots, y_n\}$, t-SNE models pairwise similarities using a **Student-t distribu-**

tion with one degree of freedom instead of a Gaussian [3]:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}} \tag{14}$$

The Student-t distribution has **heavy tails**: it assigns non-negligible probability mass to large distances. This means a moderate distance in the high-dimensional space can be faithfully represented by a *much larger* distance in the 2D map without incurring a large probability mismatch. This property algebraically dissolves the attractive force between dissimilar points, solving the crowding problem [3].

5.3 Optimization: Minimizing the KL Divergence

The map $\mathcal{Y}$ is found by minimizing the Kullback-Leibler (KL) divergence between the high-dimensional and low-dimensional probability distributions using gradient descent [3]:

$$\mathcal{L} = \mathrm{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{15}$$

The gradient has a physically intuitive form. Defining $Z = \sum_{k \neq l}(1 + \|y_k - y_l\|^2)^{-1}$, the gradient with respect to map point y_i is [3]:

$$\frac{\partial \mathcal{L}}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij})(1 + \|y_i - y_j\|^2)^{-1}(y_i - y_j) \tag{16}$$

This gradient is equivalent to a system of physics-based springs connecting each pair of map points. The spring between y_i and y_j *attracts* the points when $p_{ij} > q_{ij}$ (they are closer in the map than in the data) and *repels* them when $p_{ij} < q_{ij}$. The map points are settled into a stable configuration through iterative gradient descent with momentum [3]:

$$\mathcal{Y}^{(t)} = \mathcal{Y}^{(t-1)} + \eta \frac{\partial \mathcal{L}}{\partial \mathcal{Y}} + \alpha(t) \left(\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)}\right) \tag{17}$$

where η is the learning rate and $\alpha(t)$ is the momentum coefficient.

5.4 The t-SNE Algorithm

Algorithm 1 presents the complete t-SNE procedure as described by Van der Maaten and Hinton [3].

5.5 Limitations of t-SNE

While highly effective for 2D/3D visualization, t-SNE has critical weaknesses that limit its use as a general-purpose dimensionality reduction tool [3]:

1. **Dimensionality restriction:** t-SNE is unproven for $d > 3$ output dimensions. In higher output spaces, the heavy tails of the Student-t distribution consume a disproportionate fraction of the probability mass in Q, potentially destroying local cluster structure.

Algorithm 1: t-Distributed Stochastic Neighbor Embedding (t-SNE)

Input: Data matrix $X = \{x_1, \ldots, x_n\}$; perplexity Perp; number of iterations T; learning rate η; momentum $\alpha(t)$

Output: Low-dimensional map $\mathcal{Y}^{(T)} = \{y_1, \ldots, y_n\}$

`// Step 1: Compute high-dimensional affinities`

for *each* i **do**

  Compute $p_{j|i}$ using a Gaussian kernel centered at x_i;

  Find σ_i via binary search such that $\mathrm{Perp}(P_i) = \text{target perplexity}$;

end

Set symmetrized affinities: $p_{ij} = \dfrac{p_{j|i} + p_{i|j}}{2n}$;

`// Step 2: Initialize the map`

Sample initial map points: $\mathcal{Y}^{(0)} \sim \mathcal{N}(0, 10^{-4} \cdot I)$;

`// Step 3: Gradient descent loop`

for $t = 1$ **to** T **do**

  Compute low-dimensional affinities q_{ij} using Student-t kernel (Eq. 5);

  Compute gradient $\dfrac{\partial \mathcal{L}}{\partial \mathcal{Y}}$ (Eq. 7);

  Update map: $\mathcal{Y}^{(t)} \leftarrow \mathcal{Y}^{(t-1)} + \eta \dfrac{\partial \mathcal{L}}{\partial \mathcal{Y}} + \alpha(t)\big(\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)}\big)$;

end

return $\mathcal{Y}^{(T)}$

2. **Loss of global distances:** t-SNE faithfully preserves *local* neighborhoods, but distances between well-separated clusters in the 2D map are not interpretable. Two clusters that appear far apart may not be any more dissimilar than two that appear close.

3. **Non-convex cost function:** The KL divergence cost is not convex, so gradient descent is not guaranteed to find a global minimum. Different random initializations can produce significantly different maps, requiring multiple runs and careful parameter tuning [3].

4. **Computational cost:** The naive algorithm is $O(n^2)$ in both time and memory, making it impractical for corpora exceeding ∼10,000 documents without the Barnes-Hut approximation ($O(n \log n)$).

6 NLP Applications and Experimental Synthesis

6.1 LSA Applied to Information Retrieval: MED Corpus Experiments

The primary application of LSA is augmenting Information Retrieval systems. In experiments on the **MED dataset** (a corpus of 1,033 medical abstracts with 30 benchmark queries), the SVD-based LSA method at $k = 100$ dimensions achieved a **13% improvement in precision** over raw term-matching at high recall levels [1]. More striking still,

LSA proved capable of retrieving highly relevant documents that shared *zero literal terms* with the query—a result that is simply impossible for term-matching systems [1]. This directly validates the hypothesis that semantic structure, obscured by synonym noise in the raw matrix, is recoverable through truncated SVD.

6.2 t-SNE Applied to Semantic Visualization of Word Embeddings

In NLP, t-SNE is the standard tool for inspecting the geometry of pre-trained dense word embeddings such as Word2Vec, GloVe, and contextual embeddings from Transformer models. Unlike the sparse, count-based BoW vectors that LSA decomposes, these embeddings are dense 100–768 dimensional vectors trained to encode semantic similarity by proximity.

t-SNE's probabilistic nature excels at preserving *local semantic neighborhoods* [3]. When applied to word embeddings, it effectively untangles the high-dimensional semantic space to produce tightly clustered, visually distinct groupings—synonyms cluster together, antonyms appear near each other but on opposite sides, and semantic fields (animals, countries, professions) form clearly separated islands. This makes t-SNE indispensable for exploratory data analysis in NLP, model debugging, and validating that a training procedure has instilled the intended semantic geometry.

6.3 Comparison Summary

Table 2: Comparison of the three dimensionality reduction methods across key NLP criteria.

Criterion	**PCA**	**LSA**	**t-SNE**
Underlying math	Eigenvector decomp.	Truncated SVD	KL divergence / gradient descent
Linearity	Linear	Linear	Non-linear
Operates on	Any matrix	Term-doc matrix	Any embedding
Preserves	Global variance	Latent semantics	Local neighborhoods
Handles synonymy?	Partially	Yes	N/A (post-embedding)
Output interpretable?	Yes (PCs)	Yes (topics)	Only for 2D/3D plots
Use in IR?	Rarely	Yes	No
Use in visualization?	Limited	Limited	Standard

7 Conclusion

The challenge of extracting meaning from high-dimensional text data requires a robust, interconnected toolkit of dimensionality reduction algorithms. PCA establishes the vital mathematical baseline: it shows that the axes of maximum variance in a dataset can be

found via eigenvector decomposition of the covariance matrix, equivalently via SVD, and that projecting onto the top k of these axes constitutes an optimal linear compression.

LSA transplants this exact framework onto the term-document matrix of NLP, using truncated SVD to build a k-dimensional latent semantic space. This space is the key technical innovation: it allows terms and documents to be mapped to the same geometric coordinates and compared via cosine similarity, directly solving the synonymy and polysemy problems that defeat literal term-matching. Experimental results on the MED corpus confirm a measurable, consistent improvement in precision.

Finally, t-SNE provides the necessary non-linear probabilistic extension for the visualization challenge that linear methods cannot solve. By replacing low-dimensional Gaussian kernels with a Student-t distribution, it algebraically neutralizes the crowding problem, producing the clean, semantically faithful cluster maps that make high-dimensional word embeddings interpretable to human researchers.

Together, these three methods form a comprehensive pipeline for understanding and processing natural language: BoW $\rightarrow$ TF-IDF $\rightarrow$ LSA (for retrieval and compression) $\rightarrow$ t-SNE (for visualization and inspection).

Acknowledgement: This work is the result of my own activity, and I confirm I have neither given, nor received unauthorized assistance for this work. I declare that I used generative AI or automated tools in the creation of content or drafting of this document.

References

[1] Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). *Indexing by latent semantic analysis. Journal of the American Society for Information Science*, 41(6), 391–407.

[2] Shlens, J. (2014). *A tutorial on principal component analysis.* arXiv preprint arXiv:1404.1100.

[3] Van der Maaten, L., & Hinton, G. (2008). *Visualizing data using t-SNE. Journal of Machine Learning Research*, 9(11), 2579–2605.